

MEDHAVA

Gap analysis

What is missing, what it holds up, and which gaps can be closed from inside this repository at all.

The other audit documents each measure one register. This one is the joins between them, which is where the useful findings live: a capability nobody started matters differently depending on whether four other things wait behind it.

Gap 1 — the largest one, stated plainly

146 of 165 apps are written down and not standing up. That is not a defect and it is not a surprise — a design is meant to be ahead of the build — but it is the number every other figure in this project should be read against. The design describes a business operating system; what runs today is 19 apps on the real database and 0 browser prototypes over an in-page store.

Gap 2 — nothing has ever been deployed, and it blocks the most

CAP-DEPLOY is NOT STARTED. Behind it:

- **CAP-MONITOR** — there is nothing to monitor until something runs somewhere
- every **PRODUCTION-READY** rung in the requirements registry, all of them empty
- the whole top of the 0–5 score: no row can reach 5
- the maturity level, which the gate caps at Prototype while this holds

And it cannot be closed from inside this environment. It needs a server, a domain and credentials, and the egress proxy here refuses everything but package registries and GitHub. This is the clearest example of a gap that is a **decision and a purchase**, not a piece of work waiting to be done.

Gap 3 — sixteen apps nothing is watching

0 apps run and carry their own self-tests, and those tests are not inside `npm test`. Nothing would notice them breaking. They score 3 — "implemented but weakly verified" — for exactly that reason, and it is the only gap in this document that can be closed without writing a feature. It is `Q01` in the build queue for the same reason.

Gap 4 — capabilities nobody has started

Capability	Why it is not started	What waits behind it
<code>CAP-DEPLOY</code> Deployment to a production environment	No server, no domain and no credentials exist.	<code>CAP-MONITOR</code> , every production claim
<code>CAP-DEVPLATFORM</code> Developer platform — public API, webhooks, SDKs	Not begun.	<code>CAP-MARKETPLACE</code> , embedded analytics
<code>CAP-STUDIO</code> Studio — building screens and flows without code	Not begun.	nothing else in the register
<code>CAP-MARKETPLACE</code> Extension marketplace	Not begun, and it cannot begin before <code>CAP-DEVPLATFORM</code> : there is nothing for a third party to extend..	nothing else in the register
<code>CAP-THREATMODEL</code> Threat model and adversarial security testing	Isolation is proven against an honest client.	nothing else in the register
<code>CAP-SCALE</code> Behaviour at production scale	Every test here runs on a handful of rows.	nothing else in the register

Gap 5 — what cannot be closed by building

`CAP-PUBLISH` **The static site published at a public URL** — Blocked on one repository setting, not on any code. The site is assembled by `mksite.js` and verified by `checksite.js`, which serves it over real HTTP and drives it in Chromium — 13 assertions, recorded. The workflow that would publish it cannot: creating a GitHub Pages site needs administration rights the Actions token does not have here, and the attempt fails with "Resource not accessible by integration". Pages has never been enabled on this repository — the runs from July failed the same way. Settings → Pages → Build and deployment → Source: GitHub Actions is the whole fix, and it is the owner's to make. `enablement: true` was expected to do it unattended and does not.

`CAP-INTEGRATIONS` **Live outside integrations** — Every marketplace, courier, tax portal, bank feed and payment provider needs live credentials, and this repository must never hold one. This cannot be raised from inside the

repository at all — it needs an environment holding secrets that no commit ever sees. Simulating one and calling it connected is the failure this whole register exists to make impossible.

This is the distinction the whole registry rests on. A BLOCKED row is not slow progress; it is progress that a commit cannot make. Reporting it as "in progress" is the failure this project has been trying to make structurally impossible.

Gap 6 — specified capabilities with nothing behind them yet

Capability	What is missing
CAP-MONITOR Monitoring, alerting and health checks	Nothing to monitor until something is deployed. Depends on CAP-DEPLOY.
CAP-AI AI gateway, agent permissions and evaluation	The provider router is the one piece that runs. There is no gateway, no permission model for what an agent may do on a company's data, and no evaluation set, so no quality claim can be made about any answer the system gives.
CAP-ANALYTICS Query engine and report builder on real data	The prototype report builder computes over an in-page store. There is no query engine against the database.
CAP-MOBILE Mobile	The shell is responsive markup that has never been opened on a phone by any check here. A responsive claim nobody has tested is a claim, and this register will not count it as a result.

Gap 7 — capability classes with no app at all

The 0 lines of the owner's specification that resolve to nothing at all gather into 12 themes. **8 of those themes have no capability row behind them** — not a capability marked NOT STARTED, which is at least a decision written down, but nothing in the register whatsoever.

Theme	What it would actually mean	What it costs
Agile delivery — backlog, sprints, boards, burndown	A board with stories, points and a sprint boundary, over the tasks that module 20 already has. Thirteen lines, one screen and one report.	Self-contained and genuinely optional. Nothing else in the design depends on it, and a business running a garment factory may never want it.
Meetings — scheduling, participants, polls, the record of a call	The half of meetings that is NOT live video: who is invited, when, who came, what was decided, and a poll. The video itself is in the constraints document because it needs media infrastructure; this half needs none.	Small, and worth separating from video precisely because the useful part does not need the expensive part.
Calendar — events, invitations, availability, shared calendars	Eleven lines that are one thing. Recurring events and time zones are where the difficulty actually is, and both are solved problems with a library rather than judgement.	Moderate. Syncing to an outside calendar is in the constraints document; a calendar of its own is not.
Mail handling — folders, filters, rules, signatures, a shared inbox	The reading and organising half of email. HOSTING mail is in the constraints document — deliverability and spam reputation are infrastructure — but once a mailbox exists somewhere, filtering and routing it is ordinary work.	Only worth starting after a mail domain exists, which is a constraint, not a task.
Document applications — writing, presenting, notes, track changes	A word processor, a presentation tool and notes. Eight lines that are three substantial products, and the specification lists them as though they were features.	The worst effort-to-value ratio in the backlog. Every business already has these, and nobody adopts a business system for its word processor.
Learning — courses, lessons, assessments, progress	Course material, the lessons inside it, an assessment at the end, and a record of who has completed what. Eight lines that are one straightforward application over data module 16 already holds about who works here.	Small and genuinely useful in a factory, where a machine or a process has to be taught and the record of who was taught it matters for compliance.
Sales territories	Dividing customers and targets by region or team, and reporting against that division.	Small. Matters when there is a sales team large enough to divide, and not before.
Seven small independent lines	Time zones, a dark theme, appointment booking, employee engagement, social engagement, maps in reports, and embedded analytics. Genuinely unrelated to each other and to everything above.	Each is hours to days. They are grouped only because grouping them stops six one-line themes pretending to be structure.

The other 4 themes DO have a capability row — a low-code builder is [CAP-STUDIO](#) and a developer platform is [CAP-DEVPLATFORM](#), both NOT STARTED. Those are a different kind of gap: the surface is named and unbuilt rather than unnamed. [checkbacklog.js](#) rebuilds this list from the coverage register on every run, so a line that gets covered leaves it without anybody editing anything.

What this gap analysis is not

It is not a priority order. Which gap to close first is a decision that weighs what this project can verify against what the business needs next, and only the second half is the owner's to supply. `BUILD_QUEUE.md` proposes an order and states the reasoning for each position so it can be argued with.

Where every number here comes from

Nothing in this document is typed. It is generated by `node brand/delivery/website/mkaudit.js` and every figure is read at that moment from the register that owns it:

Fact	Register	Gate
modules and apps	<code>brand/site/modules.js</code>	<code>checkneutral.js</code> , <code>checkshape.js</code>
what each has reached	<code>brand/site/registry.js</code>	<code>checkregistry.js</code>
the 0–5 score and the queue	<code>brand/site/audit.js</code>	<code>checkaudit.js</code>
capability holes with no app	<code>brand/site/backlog.js</code>	<code>checkbacklog.js</code>
rules and their proofs	<code>brand/site/rules.js</code>	<code>checkrules.js</code>
recorded runs	<code>docs/verification/EVIDENCE.md</code>	<code>tools/evidence.js --check</code>

A figure in this document that disagrees with its register means the document is stale. Regenerate it; `npm test` refuses a stale one.

Every technical word above, in plain language

12 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

model

The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.

Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

permission

One specific thing a role is allowed to do, like approving a discount or viewing salaries.

Guchhe ki ek chaabi. Ek chaabi ek darwaza.

Medhava · One business operating system · Any industry